

# Web Service Design and Practice

## 09. Socket.io

Dr. Kyudong Park

School of Information Convergence



# Contents

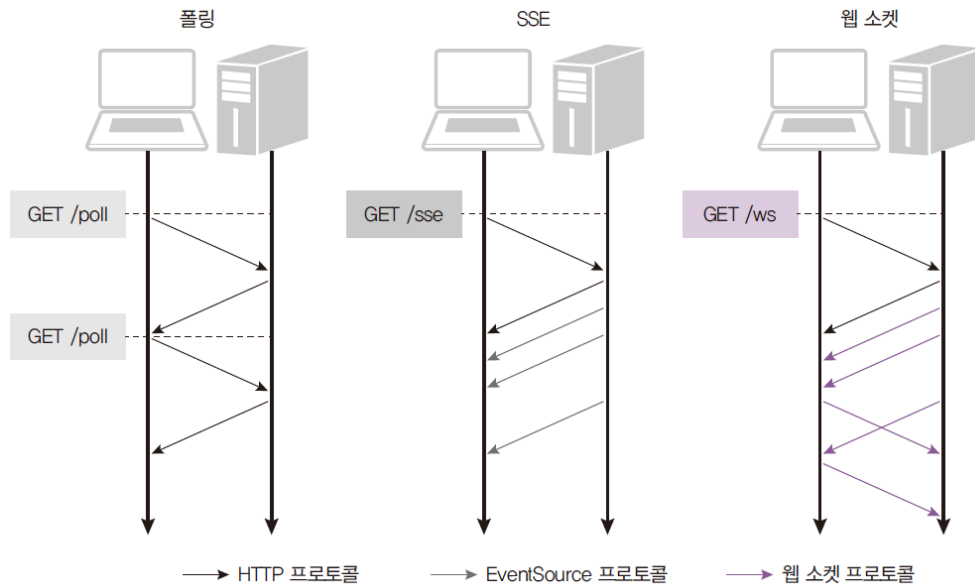
- 배경
- 웹 소켓 이해하기
- Socket.io 구현
- 채팅방 구현 실습
- 추가 Topic
  - 방 만들기
  - 서버의 주기적 이벤트 발송
  - 서버에서의 중요 변수 관리
  - Front-end 분리 시 처리

# 1. 배경

- HTTP 통신의 한계
  - Request 와 Response
  - 서버의 데이터가 update 되더라도 클라이언트가 서버로 요청을 보내지 않으면 변경된 데이터를 받아올 수 없음
    - 실시간으로 데이터를 제공해야하는 프로그램의 경우 (such as 주식) 에는 어떻게 구현을 해야하는가
      - Polling : 클라이언트에서 밀리초 단위로 끊임없이 서버에 Request
      - 사용자가 많아지면 서버에 부담!
      - 서버의 어디에 부담이 갈까...?

# 1. 배경

- SSE(Server Sent Events)
  - EventSource라는 객체를 사용
  - 처음에 한 번만 연결하면 서버가 클라이언트에 지속적으로 데이터를 보내줌
  - 클라이언트에서 서버로는 데이터를 보낼 수 없음



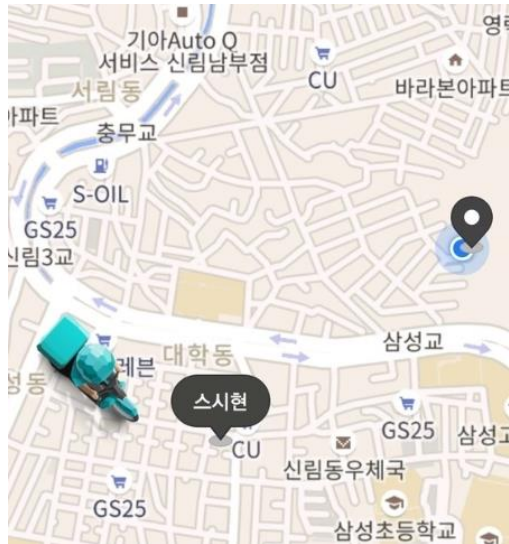
## 2. 웹 소켓 이해하기

- 웹 소켓: 실시간 양방향 데이터 전송을 위한 기술
  - 클라이언트 요청 없이도 서버에서 데이터를 전송할 수 있음
  - ws 프로토콜 사용 -> 브라우저가 지원해야 함 (최신 브라우저는 Web Socket 을 지원함)
  - Node는 ws나 Socket.IO같은 패키지를 통해 웹 소켓 사용 가능



## 2. 웹 소켓 이해하기

- 실무에서 웹 소켓의 활용 사례
  - 지도
    - 지도 상에 움직이는 특정 객체의 위치를 실시간으로 표기해야하는 경우
    - 카카오T 의 실시간 위치, 배달의 민족 라이더 위치
  - 대시보드
    - 기업 내부에서 실시간 매출 지표, 성과 지표를 보여주어야 할 때
  - 실시간 지표
    - 주식, 환율 등 실시간으로 변화하는 수치를 제공하는 경우



# 3. Socket.io 구현

- socket.io 설치

```
$ npm install socket.io
```

- socket Handler 연결
  - bin/www.js 에 직접 구현 or 별도 모듈으로 구현
    - 구현 부분을 별도로 분리하는 것이 유지보수에 좋음
  - `const socketHandler = require("../modules/socketHandler");`
  - `const server = http.createServer(app);`
  - `socketHandler(server);`

## 3. Socket.io 구현

- socket Handler 구현
  - modules 폴더 내 socketHandler.js 구현

```
const { Server } = require("socket.io");

const socketHandler = (server) => {
  const io = new Server(server);
  io.on("connection", (socket) => { // 접속 시 서버에서 실행되는 코드
    const req = socket.request;
    const socket_id = socket.id;
    const client_ip = req.headers["x-forwarded-for"] || req.connection.remoteAddress;
    console.log("connection!"); console.log("socket ID : ", socket_id);
    console.log("client IP : ", client_ip);

    socket.on("disconnect", () => { // 사전 정의 된 callback (disconnect, error)
      console.log(socket.id, "client disconnected");
    });

    socket.on("event1", (msg) => { // 생성한 이벤트 이름 event1 호출 시 실행되는 callback
      console.log(msg);
    });
  });
};

module.exports = socketHandler;
```



## 3. Socket.io 구현

- socket Handler 구현 (server)
  - 이벤트 리스너 등록 : `socket.on("event_name", () => {})`
  - 메시지 전달 (클라이언트에게)
    - `io.emit('event_name', data)` : 접속된 모든 클라이언트로 메시지 전송
    - `socket.emit('event_name', data)` : 메시지를 전송한 클라이언트에게만 메시지 전송
    - `socket.broadcast.emit('event_name', data)` : 메시지를 전송한 클라이언트를 제외한 나머지 모든 클라이언트에게 메시지 전송
    - `io.to(id).emit('event_name', data)` : 지정된 특정 클라이언트 ID 에게만 메시지 전송

## 3. Socket.io 구현

- index.html 구현 (client)

- 접속

```
const socket = io();
socket.on("connect", () => {
  // 최초 접속 시, 구동됨
  socket.emit("event1", "hi");
});
```

- 메시지 전달

- socket.emit("event\_name", data) : 서버로 data 전송

### 3. Socket.io 구현

- 서버와 클라이언트의 전송 개념도
  - 클라이언트는 둘 이상 될 수 있다는 사실을 유념
    - 서버는 메시지를 어떤 클라이언트로 보내줄 것인지 잘 설계해야함

```
const socketHandler = (io) => {
  io.on("connection", (socket) => {
    // 중략
    socket.on("event1", (msg) => {
      socket.emit("getID", { id: socket_id });
      socket.broadcast.emit("new", { text: "new user!" });
      console.log(msg);
    });
  });
};

module.exports = socketHandler;
```

```
const socket = io();
socket.on("connect", () => {
  // 최초 접속 시, 구동됨
  socket.emit("event1", "hi");
});
```

```
const socket = io();
socket.on("connect", () => {
  // 최초 접속 시, 구동됨
  socket.emit("event1", "hi");
});
```

### 3. Socket.io 구현 - 동작 확인

- 개발자 도구 Network 탭을 보면 웹소켓과 폴링 연결 둘 다 있음 확인 가능

|                                                     |     |         |                |       |        |  |
|-----------------------------------------------------|-----|---------|----------------|-------|--------|--|
| <input type="checkbox"/> ?EIO=4&transport=pollin... | 200 | xhr     | polling-xhr... | 261 B | 1 ms   |  |
| <input type="checkbox"/> ?EIO=4&transport=pollin... | 200 | xhr     | polling-xhr... | 149 B | 3 ms   |  |
| <input type="checkbox"/> ?EIO=4&transport=pollin... | 200 | xhr     | polling-xhr... | 196 B | 3 ms   |  |
| <input type="checkbox"/> ?EIO=4&transport=webs...   | 101 | webs... | websocket...   | 0 B   | Pen... |  |
| <input type="checkbox"/> ?EIO=4&transport=pollin... | 200 | xhr     | polling-xhr... | 164 B | 115... |  |



- Socket.IO는 먼저 폴링 방식으로 연결 후(웹 소켓을 지원하지 않는 브라우저를 위해), 웹 소켓을 사용할 수 있다면 웹 소켓으로 업그레이드
- 웹 소켓만 사용하고 싶다면 transports 옵션을 다음과 같이 주면 됨

index.html

```
...
<script>
  const socket = io.connect('http://localhost:8005', {
    path: '/socket.io',
    transports: ['websocket'],
  });
```

## 4. 채팅방 구현 실습

- 채팅방 구현 실습
  - ID 표시
  - 메시지 수신
    - 상대방의 ID 표시
  - 메시지 송신
    - 모두에게
    - 본인 제외
    - 특정 사용자에게 (ID 명시)

MY ID: P-7JZ-46yKguGzwvAAAT

P-7JZ-46yKguGzwvAAAT에서 보낸 메시지 : 안녕하세용

r9Uvye3zNCmg0WmbAAAV에서 보낸 메시지 : ㅋㅋㅋ 네 반가워요

## 5. 방 만들기

- 동일 사용자 그룹으로 간주
  - 같은 그룹에 속한 socket 들에 대해 한번에 이벤트를 발송할 수 있음
  - 채팅방을 여러 개 만들어서 운영한다고 할 때 유용함

```
socket.on("joinRoom", function (msg) {  
  let roomName = msg;  
  socket.join(roomName);  
});  
  
socket.on("chatting", function (msg) {  
  io.to(msg.roomName).emit("broadcast", msg.msg);  
});
```

## 6. 서버의 주기적 이벤트 발송

- 사전 정의된 시간이 흐를 때마다 클라이언트로 이벤트 발송
  - 사례 : 전체 클라이언트의 시간을 조율하고자 할 때 (게임 등)

```
let isStop = false;
let counter = 10;

socket.on("countdownbtn", (id, data) => {
  isStop = false;
  counter = 10;
  const cdb = setInterval(() => {
    if (!isStop) {
      if (counter == 0) {
        console.log("턴 종료!");
        clearInterval(cdb);
      } else {
        counter--;
        socket.emit("countdown", { number: `${counter}` });
      }
    } else {
      clearInterval(cdb);
    }
  }, 1000);
});

socket.on("countdownstopbtn", (id, data) => {
  isStop = true;
});
```

## 7. 서버에서의 중요 변수 관리

- 상태와 관련된 정보의 저장
  - js 객체 활용 권장
  - 무엇이 서버에서 관리되어야할지 고민이 필요함

```
let user = {};  
  
io.on("connection", (socket) => {  
  
  // 종락  
  
  user[socket_id] = { name: "name", point: 0 };  
  
  // 종락  
  
  socket.on("win", () => {  
    user[socket_id].point++;  
    console.log(user);  
  });  
  
  socket.on("disconnect", () => {  
    delete user[socket.id];  
  });  
  
});
```



## 8. Front-end 분리 시 처리

- socket.io 처리
  - `<script src="/socket.io/socket.io.js"></script>`
    - 이 코드는 node 에서 제공해주는 형태임
  - CDN 으로 변경 필요
    - <https://socket.io/docs/v4/client-installation/>
  - `npm install socket.io-client`
- server CORS 처리

```
const io = new Server(server, {
  cors: {
    origin: "http://localhost:3000", // 허용할 host
    methods: ["GET", "POST"], // 허용할 method
  },
});
```

# Thank you

---

Q & A